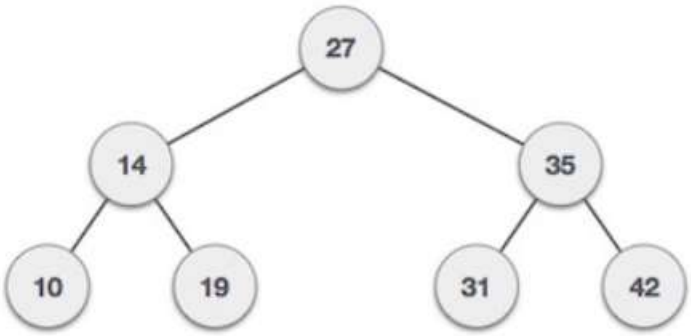
Lecture: 10
Binary Search Trees
(Part-I)

Suppose T is a binary tree. Then T is called a binary search tree if each node N of T has following property:

"The value of N is greater than every value in the left sub-tree of N and is less than every value in the right sub-tree of N."

Following is a pictorial representation of BST:



Here, root node key(27) has all less-valued keys on the left sub-tree and the higher valued keys on the right sub-tree.

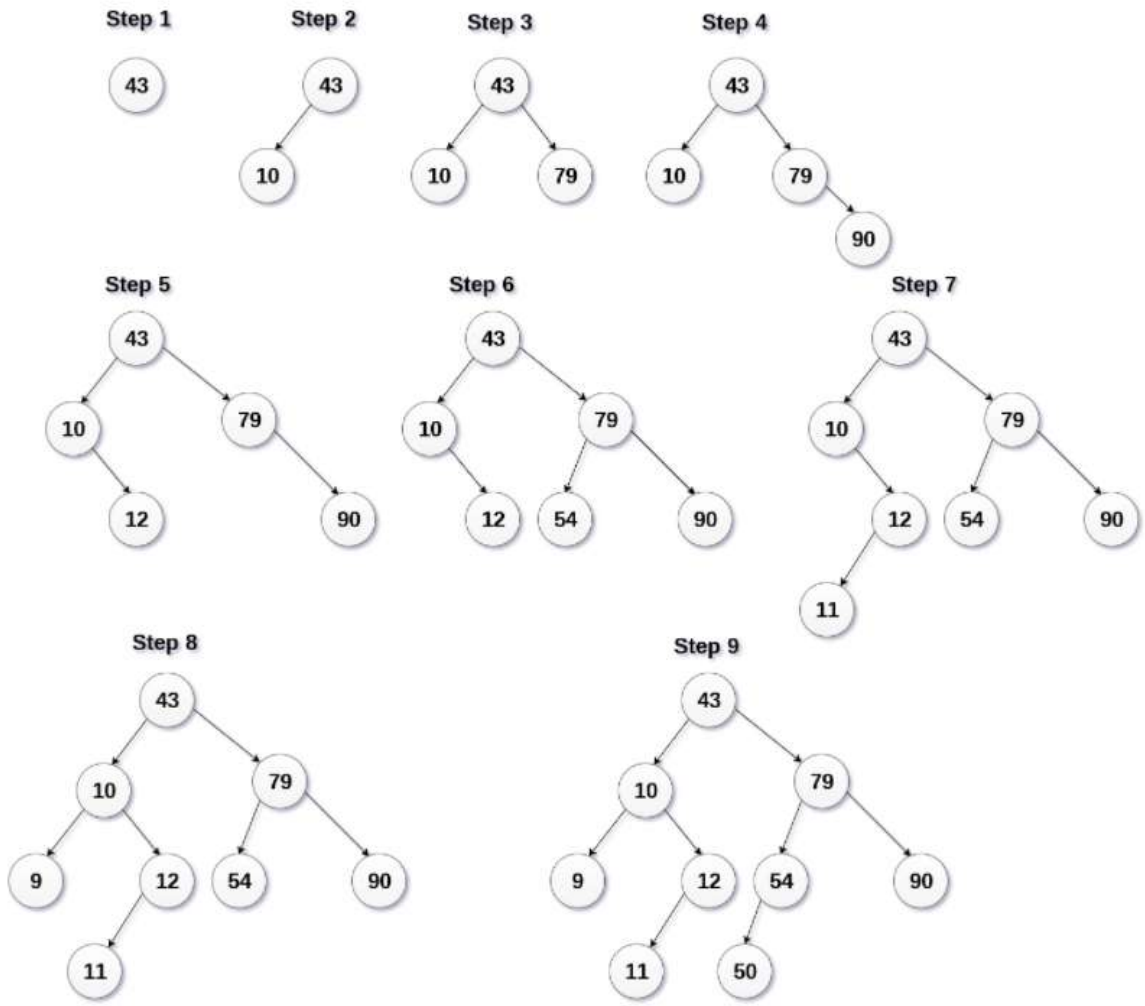
Advantages of using Binary Search Tree(BST):

- Searching become very efficient in a binary search tree since, we get a hint at each step, about which sub-tree contains the desired element.
- The binary search tree is considered as efficient data structure in compare to arrays and linked lists. In searching process, it removes half sub-tree at every step.
    - Searching for an element in a binary search tree takes $O(\log_2 n)$ time. In worst time, the time it takes to search an element is O(n).

Create the BST using following data elements:

43,10,79,90,12,54,11,9,50

- Insert 43 into the tree as the root of the tree.
- Read the next element, if it is lesser than the root node element, insert it as the root of the left sub-tree.
- Otherwise, insert it as the root of the right of the right sub-tree.



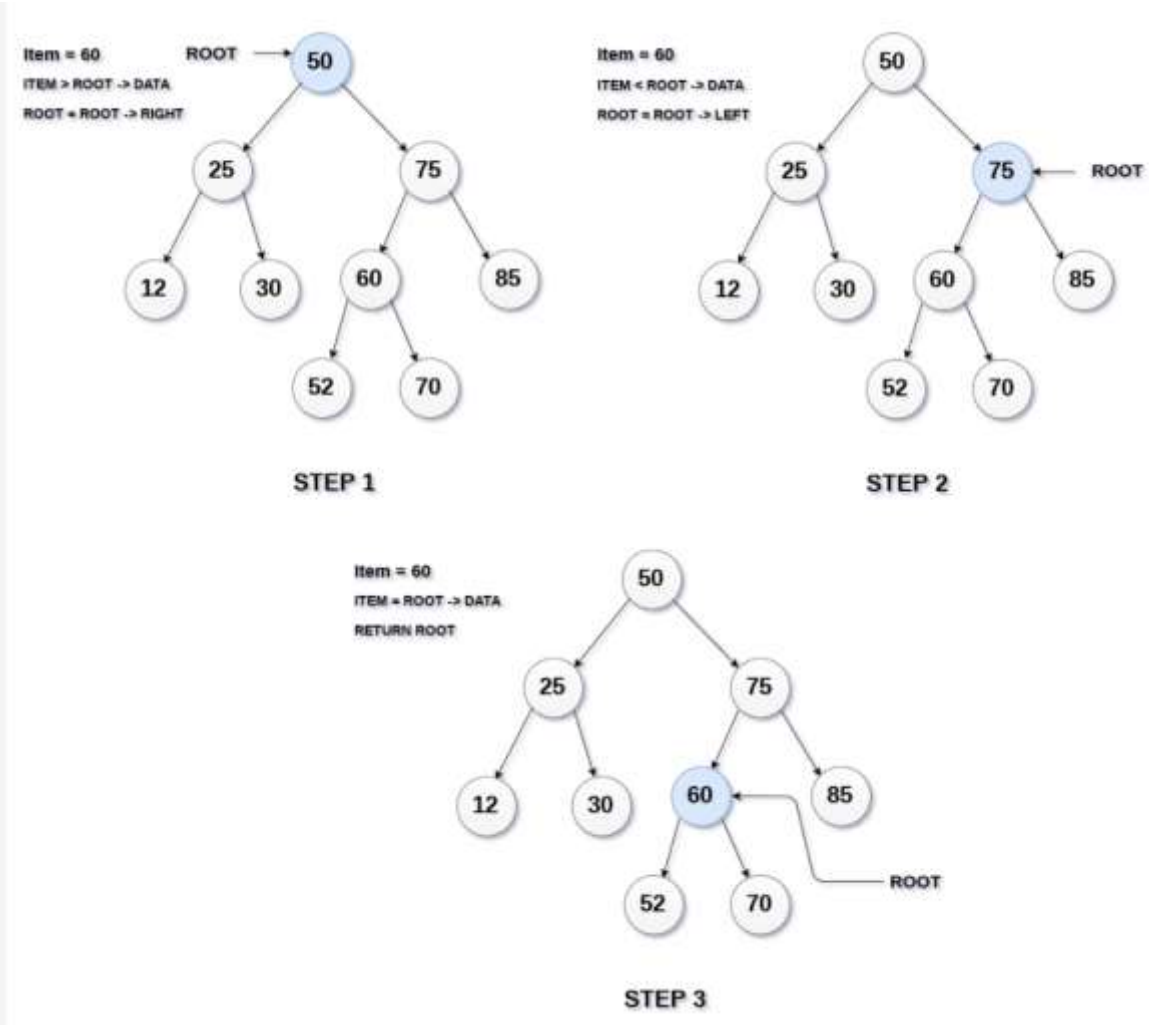
"In-order traversal of Binary Search Tree will always yield a sorted list."

Search Operation:

Searching means finding or locating some specific element or node within a data structure. Searching for some specific node in binary search tree is pretty easy due the fact that, **element in BST are stored in a particular order**.

1. Compare the element with the root of the tree.
2. If the item is matched then return the location of the node.
3. Otherwise check if item is less than the element present on root, if so then move to the left sub-tree.
4. If not, then move to the right sub-tree.
5. Repeat this procedure recursively until match found.

6. If element is not found then return NULL.



Algorithm:

```
If root == NULL
    return NULL;
If number == root->data
    return root->data;
If number < root->data
    return search(root->left)
If number > root->data
    return search(root->right)
```